

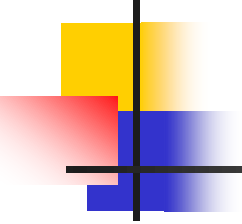


برنامه سازی پیشرفته

مبانی پایتون - 1

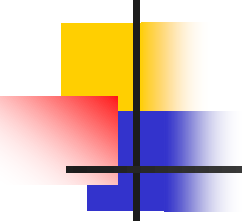
دانشگاه آزاد اسلامی
واحد تهران مرکزی

مدرس : توحید موسی زاده



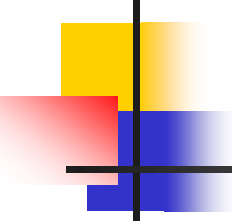
پایتون برای مهندسی برق (1)

- اکوسیستم علمی و مهندسی قدرتمند
 - NumPy / SciPy: محاسبات عددی، جبر خطی، حل معادلات دیفرانسیل
 - SymPy: محاسبات نمادین (تحلیل مدار)
 - Pandas: پردازش داده‌های اندازه‌گیری و لاگ‌ها
- شبیه‌سازی و مدل‌سازی سیستم‌های قدرت
 - PyPSA / pandapower: پخش بار، تحلیل اتصال کوتاه، بهینه‌سازی شبکه
 - SimPy: شبیه‌سازی رویداد-محور سیستم‌های کنترلی
- پردازش سیگنال و کنترل
 - Control: طراحی کنترل‌کننده، تحلیل پایداری، پاسخ فرکانسی
 - scikit-dsp-comm: طراحی فیلتر، FFT، پردازش سیگنال دیجیتال
 - MNE / librosa: پردازش سیگنال‌های واقعی



پایتون برای مهندسی برق (2)

- یادگیری ماشین و هوش مصنوعی
 - scikit-learn / TensorFlow / PyTorch
 - کاربردها: پیش‌بینی بار، تشخیص خطا، نگهداری پیش‌بینانه، بینایی ماشین صنعتی
- اتوماسیون، IoT و اتصال به سخت‌افزار
 - PySerial / pyvisa: ارتباط با ابزارهای آزمایشگاهی (اوسیلوسکوپ، مولتی‌متر)
 - Raspberry Pi / MicroPython: پیاده‌سازی روی سخت‌افزار
 - MQTT / OPC-UA: پروتکل‌های صنعتی و IIoT
- سادگی، سرعت توسعه و جامعه کاربری
 - تمرکز روی مفهوم مهندسی نه سینتکس
 - کتابخانه‌های متن‌باز و مستندات غنی
 - پل ارتباطی بین MATLAB، C/C++ و HDL



اهداف: آشنایی با محیط، متغیرها، عملگرها، ورودی/خروجی

■ آشنایی با محیط پایتون

- نصب پایتون و راه اندازی VS Code

■ متغیرها و انواع داده

- `int`, `float`, `complex`, `bool`, `str` انتساب و نام‌گذاری

■ عملگرها

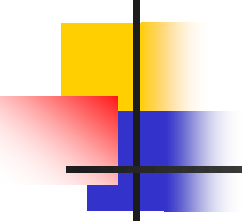
- حسابی، مقایسه‌ای، منطقی، انتسابی

■ ورودی/خروجی

- `print`, `input`، قالب‌بندی خروجی با `f-string`

■ نوشتن یک برنامه کوچک محاسباتی

- محاسبه توان یا امپدانس



محیط توسعه VS Code

■ نصب Python

■ از python.org ← گزینه Add Python to PATH را فعال کنید.

■ نصب VS Code

■ از code.visualstudio.com

■ نصب افزونه Python

■ از Microsoft شامل Pylance و Debugger

■ انتخاب مفسر

■ Ctrl+Shift+P → Python: Select Interpreter

■ اجرای سریع

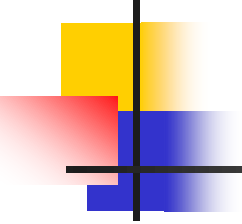
■ Ctrl+R

■ ترمینال داخلی

■ Terminal → New Terminal

■ اجرای خطبه خط

■ با Shift+Enter



عبارت‌ها و دستورات

■ Expression (عبارت) مقدار برمی‌گرداند

- کدی که به یک مقدار مشخص ارزیابی می‌شود.

- $V1 + V2$ جمع دو ولتاژ

- R * ا قانون اهم

- "AC" رشته متنی

■ Statement (دستور) عملیاتی انجام می‌دهد

- دستوری که کاری انجام می‌دهد (معمولاً شامل عبارت است).

- $V_source = 120$ دستور انتساب

- `print(P_total)` دستور چاپ خروجی

■ برنامه از مجموعه‌ای از دستورات تشکیل شده است.

- قانون: هر دستور در یک خط جداگانه نوشته شود.

- نقطه‌ویرگول (;)

- امکان نوشتن چند دستور در یک خط را می‌دهد.

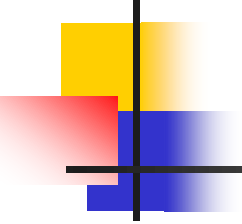
- توصیه: برای خوانایی بهتر، کمتر استفاده کنید.

کامنت‌ها

- در پایتون، هر متنی که بعد از علامت # (هشتگ) قرار گیرد، توسط کامپایلر کاملاً نادیده گرفته می‌شود.
- کاربرد: ثبت توضیحات برای خوانایی بهتر کد

■ انواع کامنت در کد

- کامنت خط کامل (توضیح برای کل خط بعد):
- محاسبه افت ولتاژ در کابل انتقال نیرو #
- کامنت درون خطی (توضیح برای همان خط کد):
- ولتاژ ورودی شبکه بر حسب ولت # $V_{in} = 220$
- مقاومت بار بر حسب اهم # $R_{load} = 4700$



تورفتگی (Indentation)

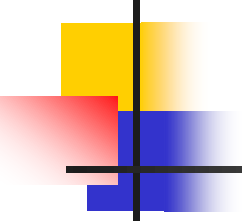
- برخلاف برخی زبان‌های برنامه‌نویسی، در پایتون فاصله‌ها (Whitespace) و تورفتگی کاملاً معنادار هستند.
- قانون طلایی: تورفتگی تصادفی مجاز نیست. تمام خطوطی که به یک بلوک تعلق دارند، باید دقیقاً همان‌دازه تورفتگی داشته باشند
- استاندارد تورفتگی: ۴ فاصله یا یک Tab
- کاربرد: مشخص کردن مرز بلوک‌های کنترلی (مثل حلقه‌ها و شرط‌ها)، توابع و کلاس‌ها.

حالت درست

```
V_in = 240
if V_in > 230:
    print("Overvoltage warning!")
    trip_breaker() # belong to if block
```

حالت اشتباه

```
V = 220
print(V)
```



متغیرها (1)

■ تعریف متغیر

■ با استفاده از عملگر انتساب = یک مقدار را به یک نام اختصاص می‌دهیم.

■ مثال رشته: `component = "Capacitor"`

■ مثال عدد اعشاری: `voltage = 12.5`

■ مثال عدد صحیح: `resistance = 4700`

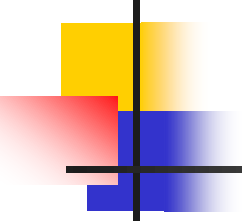
■ قوانین نام‌گذاری

■ شامل حروف، اعداد و زیرخط (`_`) باشد.

■ نباید با عدد شروع شود.

■ نباید کلمه کلیدی پایتون باشد (مثل `for`, `if`, `while`, `import`).

■ نکته: نیازی به حفظ کردن نیست؛ پایتون خودش خطا می‌دهد!



متغیرها (2)

■ نام‌های مجاز

- V_out استفاده از زیرخط، خوانا
- R_load شروع با حرف
- I_peak1 شامل عدد، ولی نه در ابتدا
- _temp شروع با زیرخط
- freq_Hz واحد مهندسی در نام

■ نام‌های غیرمجاز

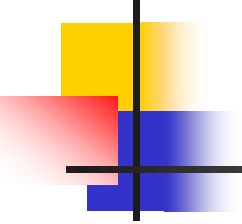
- 1st_node خطا: شروع با عدد
- V_in! خطا: کاراکتر غیرمجاز !
- R@load خطا: کاراکتر غیرمجاز @
- for خطا: کلمه کلیدی رزرو شده

متغیرها (3)

■ تشبیه مهندسی

■ متغیر = خازن، مقدار = بار الکتریکی ذخیره شده

- متغیر: ظرفی برای نگهداری داده در حافظه
- انتساب: $\text{voltage} = 220$ مثلا شارژ خازن
- تغییر مقدار: $\text{voltage} = 110$ مثلا تخلیه و شارژ مجدد
- نوع دهی پویا: نوع متغیر خودکار بر اساس مقدار تعیین می شود
- انواع پرکاربرد: `int, float, str, bool, complex`
- نام گذاری: بدون فاصله، شروع با حرف یا _، حساس به حروف بزرگ و کوچک

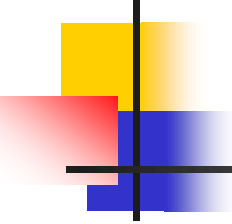


انواع داده (Data Types)

نوع	کاربرد	مثال
int	تعداد حلقه‌ها، شمارنده	$n = 5$
float	ولتاژ، جریان، توان	$V = 220.5$
str	نام سنسور، برچسب	<code>name = "PT100"</code>
bool	وضعیت کلید: روشن/خاموش	<code>is_on = True</code>

■ برای امپدانس مختلط از نوع complex می توان استفاده کرد

■ : $Z = 3 + 4j$



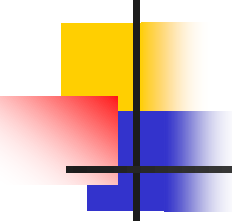
عملگرها (Operators)

- انتساب (Assignment): مقداردهی به متغیر.
- حسابی (Arithmetic): عملیات ریاضی پایه.
- مقایسه‌ای (Comparison): مقایسه دو مقدار.
- منطقی (Logical): ترکیب شروط
- بیتی (Bitwise) عملیات روی بیت‌ها
- عملگرهای خاص:
 - is : بررسی هویت
 - in : بررسی عضویت

عملگرهای ریاضی

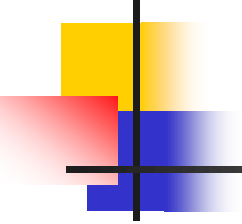
عملگر	کاربرد	مثال مهندسی
+ - * /	جمع، تفریق، ضرب، تقسیم	$V = I * R$
**	توان	$P = V^{**2} / R$
//	تقسیم صحیح	$n = 17 // 5 \# 3$
%	باقی‌مانده	$rem = 17 \% 5 \# 2$

- در پایتون ۳
- / همیشه اعشاری برمی‌گرداند؛ برای تقسیم صحیح از // استفاده کنید
- اولویت عملگرها:
- $() \leftarrow ** \leftarrow * \leftarrow / \leftarrow // \leftarrow \% \leftarrow + \leftarrow -$



عملگرهای مقایسه‌ای

- این عملگرها دو مقدار را با هم مقایسه کرده و همیشه یک نتیجه بولی True یا False برمی‌گردانند
- لیست عملگرها:
 - == : مساوی با
 - != : نامساوی با
 - < : بزرگتر از
 - > : کوچکتر از
 - <= : بزرگتر یا مساوی با
 - >= : کوچکتر یا مساوی با



عملگرهای منطقی

- عملگرهای اصلی:

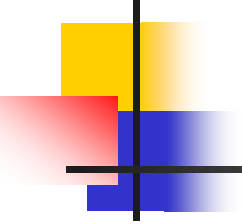
- not ■

- and ■

- or ■

- کاربرد: ارزیابی شروط در حلقه‌ها و پیاده‌سازی منطق کنترل

```
sw1 = True      # کلید بسته
sw2 = False     # کلید باز
sw1 and sw2     # False (معادل اتصال سری کلیدها)
sw1 or sw2      # True  (معادل اتصال موازی کلیدها)
```



عملگرهای بیتی

■ عملگرهایی که مستقیماً روی بیت‌ها و نمایش باینری اعداد کار می‌کنند.

■ لیست عملگرها:

■ & : AND بیتی

■ | : OR بیتی

■ ^ : XOR بیتی

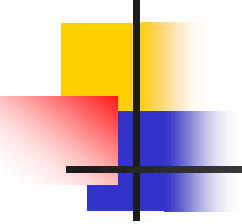
■ ~ : NOT بیتی (مکمل)

■ >> : شیفت به چپ

■ << : شیفت به راست

کاربرد در میکروکنترلرها و ارتباطات دیجیتال:

- تنظیم یک بیت (Set با OR): $PORTB = PORTB | 0x04$
- پاک کردن یک بیت (Clear با AND): $PORTB = PORTB \& \sim 0x04$
- تغییر وضعیت یک بیت (Toggle با XOR): $PORTB = PORTB \wedge 0x04$
- ماسک کردن (Masking با AND): $data = sensor_val \& 0x0F$
- شیفت (Shift - جایگزین ضرب/تقسیم): $val = ADC_raw \gg 2$



دریافت ورودی (Input)

input ■

■ همیشه رشته (str) برمی‌گرداند – حتی اگر کاربر عدد وارد کند!

■ تبدیل نوع لازم است:

■ `float(input(...))`

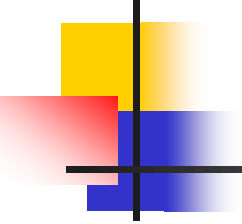
■ `int(input(...))`

■ `complex(input(...))`

مثال: ولتاژ، جریان، مقاومت

مثال: تعداد حلقه، شمارنده

مثال: امپدانس مختلط مثل $3+4j$



قالب بندی خروجی (f-strings)

■ ساختار کلی

■ `print(f"متن {متغیر} متن")`

■ تعیین دقت اعشار

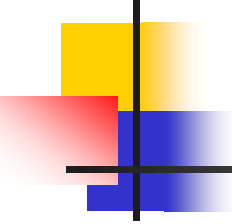
■ `print(f"Voltage: {v:.2f} V")`

■ `.2f` ← دو رقم اعشار

■ `.0f` ← بدون اعشار

■ `.3f` ← سه رقم اعشار

■ `.2e` ← نماد علمی



مثال: محاسبه ی توان، فرکانس و انرژی

■ توان

$$P=V \times I \quad \blacksquare$$

■ فرکانس و دوره تناوب

$$T=1/f \quad \blacksquare$$

■ انرژی

$$E = P \times t \quad \blacksquare$$

```
V = float(input("Voltage (V): "))  
I = float(input("Current (A): "))  
f = float(input("Frequency (Hz): "))  
t = float(input("Time (s): "))
```

```
P = V * I
```

```
T = 1 / f
```

```
E = P * t
```

```
print(f"Power: {P:.2f} W")  
print(f"Period: {T:.4f} s")  
print(f"Energy: {E:.2f} J")
```

■ هدف: تمرین input، تبدیل نوع، عملگرها و f-string

- محاسبه مقاومت معادل دو مقاومت موازی
- تبدیل ستاره به مثلث (فرمول‌های ساده‌شده)
- اگر مقاومت‌های ستاره R_a, R_b, R_c باشند:

$$R_{AB} = \frac{R_a R_b + R_b R_c + R_c R_a}{R_c}$$

$$R_{BC} = \frac{R_a R_b + R_b R_c + R_c R_a}{R_a}$$

$$R_{CA} = \frac{R_a R_b + R_b R_c + R_c R_a}{R_b}$$

- ۳ خطای عمدی در کد زیر پیدا کنید:
- راهنما: به تبدیل نوع، عملگرها و سینتکس دقت کنید.

```
V = float(input("Voltage (V): "))  
I = input("Current (A): ")      # خطا ۱  
P = V * I  
print("Power: " + P + " W")    # خطا ۲  
if P > 100  
    print("High power!")       # خطا ۳
```

■ ساخت برنامه محاسبه مصرف انرژی لوازم خانگی

- برنامه‌ای بنویسید که برای یک وسیله برقی، مصرف انرژی روزانه و ماهانه را محاسبه کند. خروجی را با f-string و دقت دو رقم اعشار نمایش دهید.

ورودی‌ها:

- نام وسیله (مثلاً یخچال، کولر، لامپ)
- توان وسیله بر حسب وات (W)
- مدت استفاده در روز بر حسب ساعت (h)

فرمول‌ها:

- انرژی روزانه $E_{day} = \frac{P \times t}{1000}$: کیلووات ساعت
- انرژی ماهانه $E_{month} = E_{day} \times 30$